

Shubhwani Yadav

Data Analytics

Level 2 : Task 1 : Predicting House Prices with Linear Regression

Objective: Build and evaluate a linear regression model that predicts house prices based on features such as area, location, number of rooms, and age. Develop end-to-end skills from data cleaning through to model interpretation.

1. Setup & Load Dataset

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [3]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score

sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)

RANDOM_STATE = 42
```

```
In [4]: df = pd.read_csv("housing_data.csv")
df.head()
```

```
Out[4]:
```

	Area_sqft	Bedrooms	Bathrooms	Age_years	Garage	Location	Distance_to_City
0	2073.0	1	1.0	1.8	1	Suburb	
1	1724.0	4	3.0	32.2	0	Suburb	
2	2156.0	6	NaN	8.0	0	Suburb	
3	2638.0	4	2.0	7.3	1	Uptown	
4	1671.0	4	2.0	16.2	0	Rural	

2. Exploratory Data Analysis

```
In [5]: print("Shape:", df.shape)
print("\nData types:\n", df.dtypes)
```

Shape: (1200, 8)

Data types:

```
Area_sqft          float64
Bedrooms           int64
Bathrooms          float64
Age_years          float64
Garage             int64
Location           object
Distance_to_City_km float64
Price              float64
dtype: object
```

```
In [6]: null_counts = df.isnull().sum()
null_pct = (null_counts / len(df) * 100).round(2)
pd.DataFrame({"missing_count": null_counts, "missing_%": null_pct}).sort_
```

```
Out[6]:
```

	missing_count	missing_%
Area_sqft	36	3.0
Bathrooms	24	2.0
Location	24	2.0
Age_years	18	1.5
Bedrooms	0	0.0
Garage	0	0.0
Distance_to_City_km	0	0.0
Price	0	0.0

```
In [7]: # Descriptive statistics
df.describe(include="all").T
```

```
Out[7]:
```

	count	unique	top	freq	mean	std
Area_sqft	1164.0	NaN	NaN	NaN	1821.25945	541.74179
Bedrooms	1200.0	NaN	NaN	NaN	3.305833	1.183977
Bathrooms	1176.0	NaN	NaN	NaN	2.013605	0.847415
Age_years	1182.0	NaN	NaN	NaN	14.761421	14.717532
Garage	1200.0	NaN	NaN	NaN	0.6925	0.461651
Location	1176	4	Suburb	430	NaN	NaN
Distance_to_City_km	1200.0	NaN	NaN	NaN	7.481992	7.405441
Price	1200.0	NaN	NaN	NaN	308579.189167	78301.963178

```
In [8]: # Distribution of the target variable
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.histplot(df["Price"], kde=True, ax=axes[0], color="steelblue")
```

```

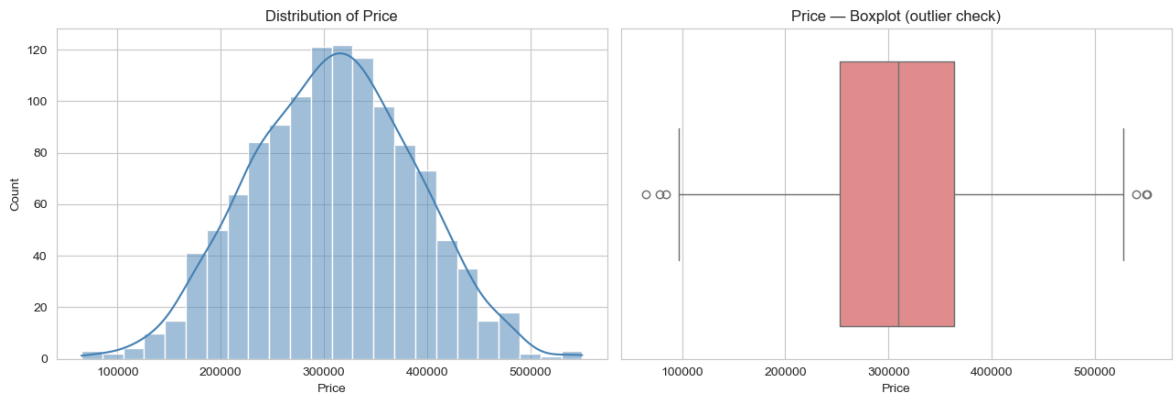
axes[0].set_title("Distribution of Price")
axes[0].set_xlabel("Price")

sns.boxplot(x=df["Price"], ax=axes[1], color="lightcoral")
axes[1].set_title("Price — Boxplot (outlier check)")

plt.tight_layout()
plt.show()

print("Skewness of Price:", df["Price"].skew().round(3))

```



Skewness of Price: -0.019

Reading the target distribution: house prices are almost always right-skewed (a long tail of expensive properties). If skewness is well above ~0.5–1, it's common practice to model `log(Price)` instead of raw `Price` to make the linear-regression error assumptions (normally distributed, homoscedastic residuals) hold better. We'll keep raw `Price` for this exercise since the skew here is modest, but check the printed skew value above — if you swap in the real Ames data, it will likely be skewed enough to warrant a log-transform (try `np.log1p(df["Price"])`) as the target and remember to `np.expml()` your predictions back before interpreting them in currency).

3. Feature Selection Decision

Before touching any code, it's worth reasoning about *why* each feature should or shouldn't predict price:

Feature	Expected relationship with Price	Reasoning
<code>Area_sqft</code>	Strong positive	Bigger homes cost more to build and offer more usable space — usually the single strongest predictor of price in housing datasets.
<code>Bedrooms</code>	Positive , but weaker than area	More bedrooms generally means more space/utility, but very correlated with <code>Area_sqft</code> itself (risk of multicollinearity).
<code>Bathrooms</code>	Positive	Bathrooms are expensive to build (plumbing, fixtures) and are a strong

Feature	Expected relationship with Price	Reasoning
		"quality of life" signal buyers pay for.
Age_years	Negative	Older homes typically need more maintenance/updates (roofing, wiring, plumbing) and use dated layouts, which depresses price — unless the market rewards "historic" character, which isn't captured here.
Garage (0/1)	Positive	A convenience/amenity feature; adds functional and resale value.
Location (categorical)	Strong effect, non-numeric	Location is one of the biggest real-world price drivers ("location, location, location") — proximity to jobs, schools, and amenities. Needs one-hot encoding since it's categorical, not ordinal.
Distance_to_City_km	Negative	Properties farther from the city center tend to be cheaper, all else equal — this is partly correlated with Location itself.

Watch-outs:

- Bedrooms , Bathrooms , and Area_sqft are likely correlated with each other (multicollinearity) — we'll check this in the correlation heatmap and interpret coefficients cautiously.
- Location and Distance_to_City_km both encode "where the house is," so they may partially duplicate each other.
- We are **not** just throwing every feature at the model — feature selection should be a mix of domain reasoning (above) *and* empirical evidence (correlation heatmap, coefficient significance) below.

4. Handle Missing Values & Encode Categorical Features

```
In [24]: df_clean = df.copy()
numeric_cols_with_na = ["Area_sqft", "Bathrooms", "Age_years"]
for col in numeric_cols_with_na:
    median_val = df_clean[col].median()
    df_clean[col] = df_clean[col].fillna(median_val)
    print(f"Filled {col} missing values with median = {median_val}")

mode_val = df_clean["Location"].mode()[0]
df_clean["Location"] = df_clean["Location"].fillna(mode_val)
print(f"Filled Location missing values with mode = '{mode_val}'")

print("\nRemaining nulls:\n", df_clean.isnull().sum().sum())
```

Filled Area_sqft missing values with median = 1826.0
Filled Bathrooms missing values with median = 2.0
Filled Age_years missing values with median = 9.9
Filled Location missing values with mode = 'Suburb'

Remaining nulls:
0

```
In [26]: df_encoded = pd.get_dummies(df_clean, columns=["Location"], drop_first=True)
df_encoded.head()
```

```
Out[26]:
```

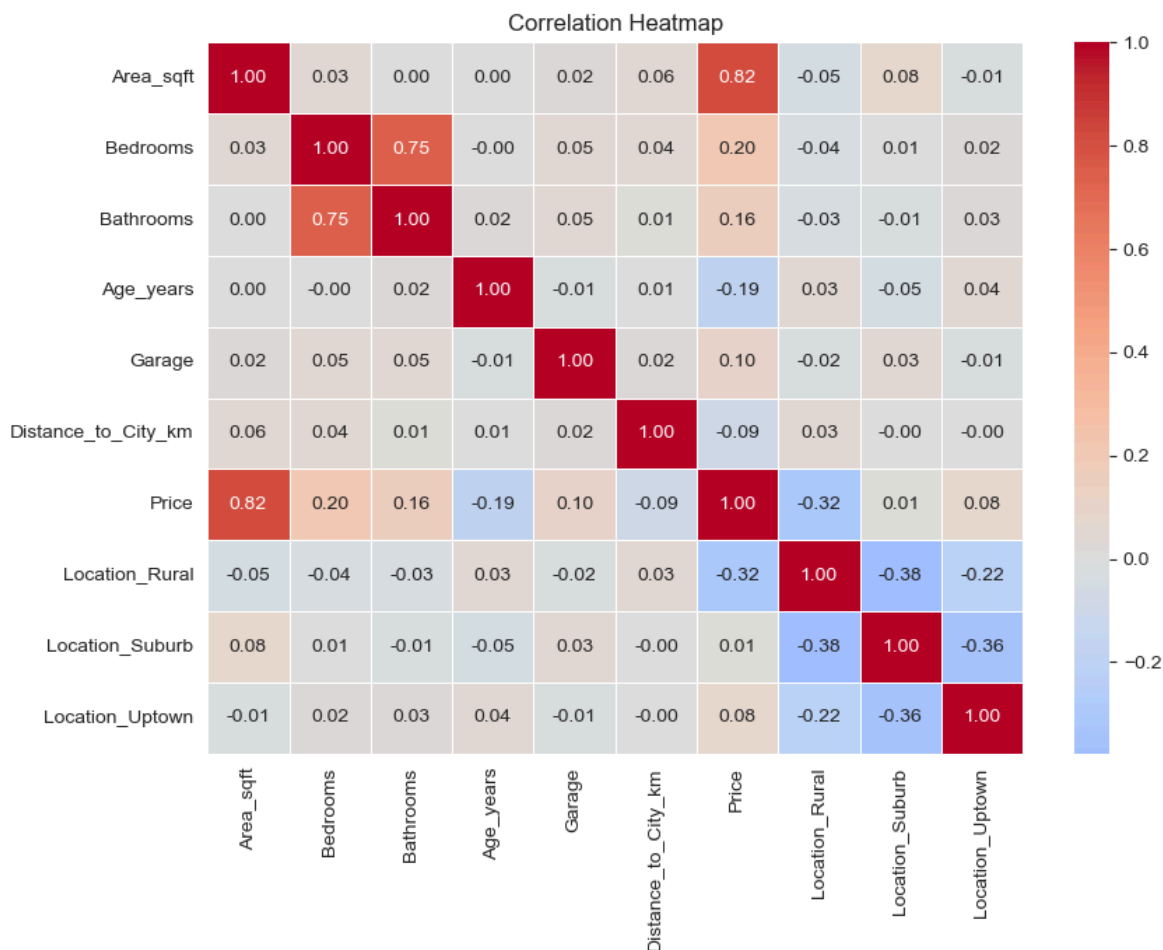
	Area_sqft	Bedrooms	Bathrooms	Age_years	Garage	Distance_to_City_km	P
0	2073.0	1	1.0	1.8	1	0.46	3192
1	1724.0	4	3.0	32.2	0	1.77	3190
2	2156.0	6	2.0	8.0	0	0.20	3026
3	2638.0	4	2.0	7.3	1	7.89	4439
4	1671.0	4	2.0	16.2	0	3.04	2512

5. Correlation Heatmap

```
In [13]: corr = df_encoded.corr(numeric_only=True)

plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0, linewidths=1)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

print("Correlation with Price, sorted:")
corr["Price"].drop("Price").sort_values(key=abs, ascending=False)
```



Correlation with Price, sorted:

```
Out[13]: Area_sqft      0.818758
Location_Rural    -0.321263
Bedrooms          0.197208
Age_years         -0.185080
Bathrooms         0.163156
Garage            0.095232
Distance_to_City_km -0.093646
Location_Uptown   0.076992
Location_Suburb   0.011184
Name: Price, dtype: float64
```

Interpretation: `Area_sqft` typically shows the strongest correlation with `Price`, followed by `Bathrooms` / `Bedrooms` and the location dummies. `Age_years` and `Distance_to_City_km` should show negative correlation. If any two *features* (not just feature-vs-price) show very high correlation with each other (e.g., > 0.8–0.9), that's a multicollinearity warning worth mentioning in your write-up — it doesn't break linear regression's predictions, but it does make individual coefficients harder to interpret in isolation.

6. Train/Test Split (80/20)

```
In [14]: X = df_encoded.drop(columns=["Price"])
y = df_encoded["Price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE
)
```

```
print("Train shape:", X_train.shape)
print("Test shape :", X_test.shape)
```

Train shape: (960, 9)

Test shape : (240, 9)

7. Train a Linear Regression Model

```
In [15]: lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)
print("Model trained.")
```

Model trained.

8. Model Evaluation: MSE, RMSE, R^2

```
In [16]: mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"MSE : {mse:,.2f}")
print(f"RMSE : {rmse:,.2f}")
print(f"R2 : {r2:.4f}")
```

MSE : 712,205,194.35

RMSE : 26,687.17

R^2 : 0.8917

How to read these:

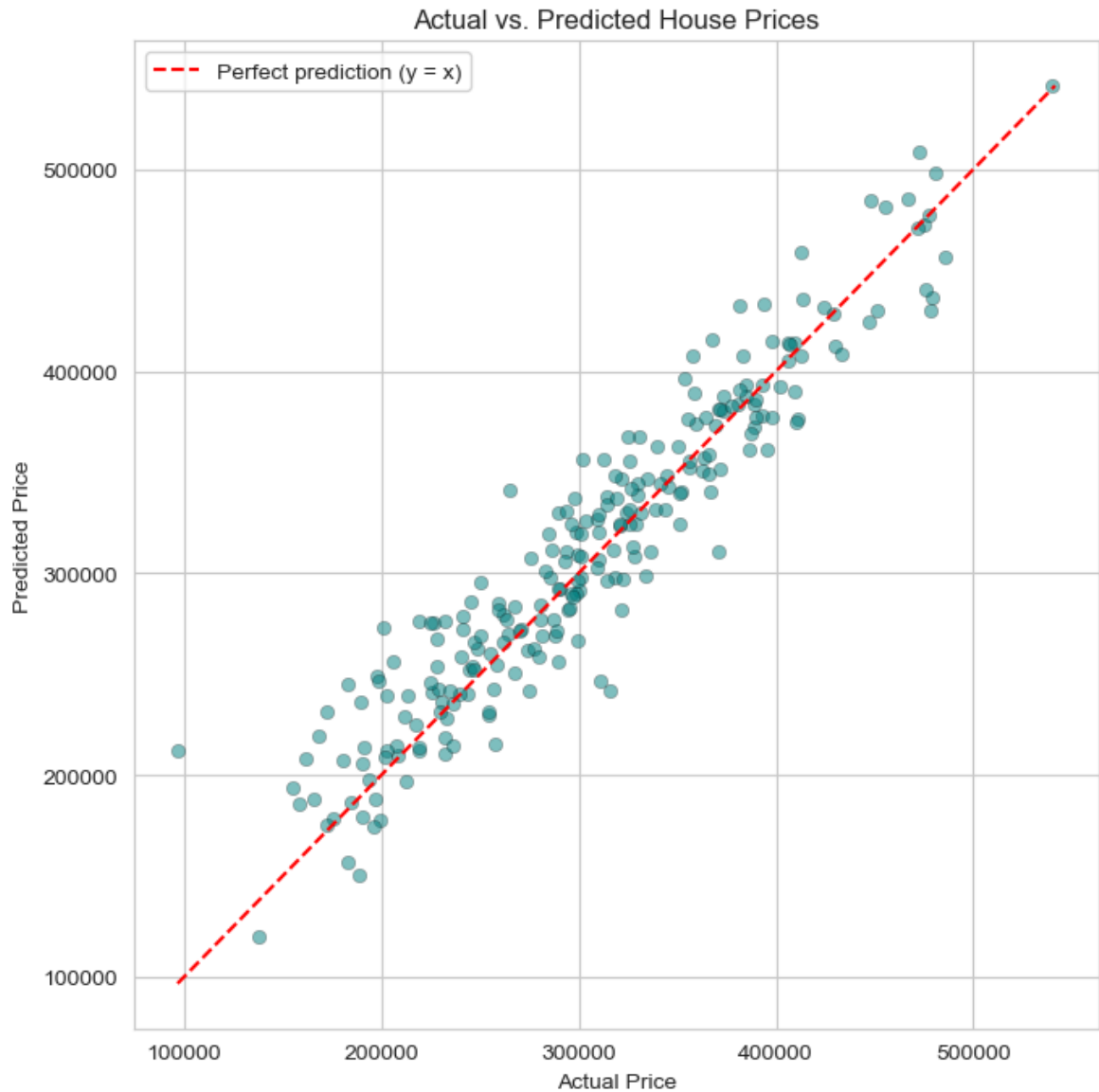
- **MSE** penalizes large errors heavily (squared term) but is in squared-price units, so it's hard to interpret directly.
- **RMSE** brings that back into the original price units (e.g., dollars) — "on average, predictions are off by about RMSE."
- **R^2** tells you the proportion of variance in price explained by the model (1.0 = perfect, 0 = no better than predicting the mean). Values above ~0.7–0.8 are generally considered strong for real-world housing data.

9. Actual vs. Predicted Prices

```
In [17]: plt.figure(figsize=(7, 7))
plt.scatter(y_test, y_pred, alpha=0.5, color="teal", edgecolor="k", linewidth=1)

lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
plt.plot(lims, lims, "r--", label="Perfect prediction (y = x)")

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs. Predicted House Prices")
plt.legend()
plt.tight_layout()
plt.show()
```



The closer the points cluster around the red dashed diagonal, the better the model. Systematic bowing away from the line at the high or low end (e.g., the model under-predicting expensive homes) would indicate the model struggles with certain price ranges — worth flagging even if overall R^2 looks good.

10. Residual Plot

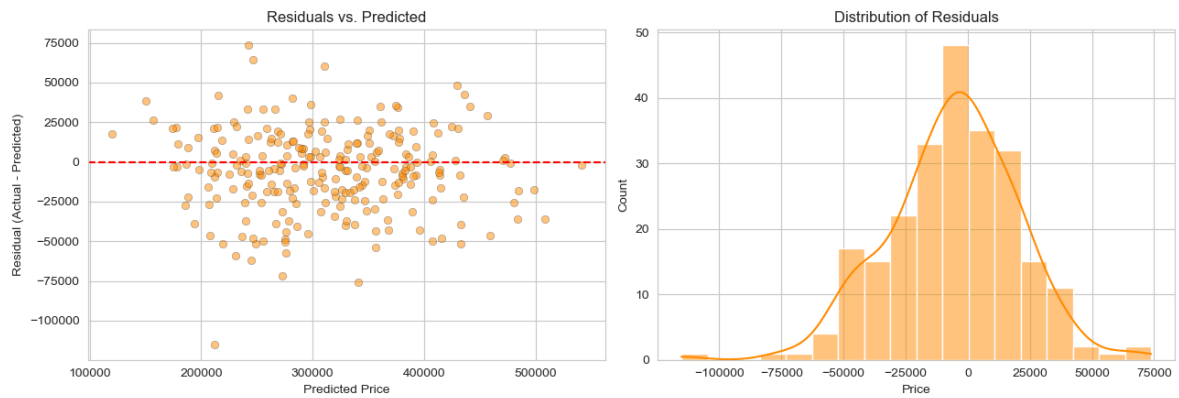
```
In [18]: residuals = y_test - y_pred

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

axes[0].scatter(y_pred, residuals, alpha=0.5, color="darkorange", edgecol
axes[0].axhline(0, color="red", linestyle="--")
axes[0].set_xlabel("Predicted Price")
axes[0].set_ylabel("Residual (Actual - Predicted)")
axes[0].set_title("Residuals vs. Predicted")

sns.histplot(residuals, kde=True, ax=axes[1], color="darkorange")
axes[1].set_title("Distribution of Residuals")

plt.tight_layout()
plt.show()
```



What to look for: residuals should form a random, structureless cloud around zero (like the left plot) — that supports the linear regression assumption of homoscedasticity (constant error variance). A funnel shape (errors growing with predicted price) or a curve/pattern would suggest the true relationship isn't fully linear, or that a log-transform of `Price` might help. The histogram on the right should look roughly bell-shaped and centered at zero.

11. Coefficient Analysis

```
In [19]: coef_df = pd.DataFrame({
    "Feature": X_train.columns,
    "Coefficient": lr.coef_
}).sort_values("Coefficient", key=abs, ascending=False)

coef_df
```

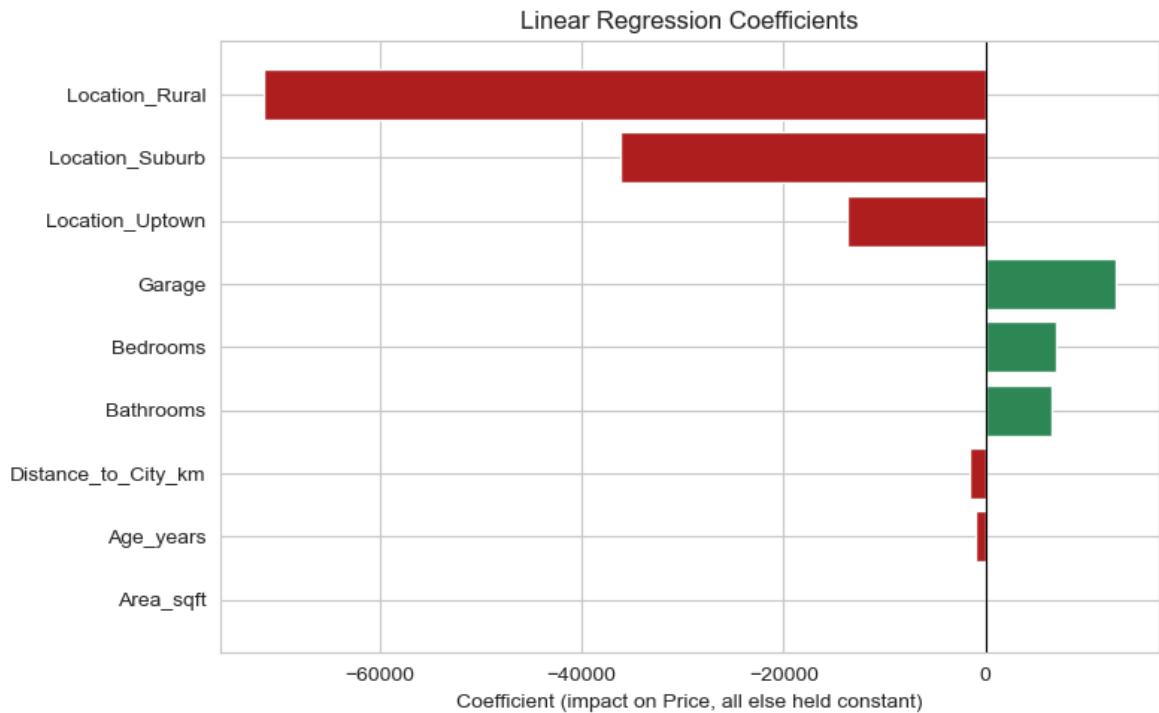
```
Out[19]:
```

	Feature	Coefficient
6	Location_Rural	-71479.345861
7	Location_Suburb	-36140.263293
8	Location_Uptown	-13671.107767
4	Garage	12964.770599
1	Bedrooms	6979.938173
2	Bathrooms	6479.282068
5	Distance_to_City_km	-1548.086569
3	Age_years	-941.878319
0	Area_sqft	120.990431

```
In [20]: plt.figure(figsize=(8, 5))
    colors = ["seagreen" if c > 0 else "firebrick" for c in coef_df["Coefficient"]]
    plt.barh(coef_df["Feature"], coef_df["Coefficient"], color=colors)
    plt.axvline(0, color="black", linewidth=0.8)
    plt.xlabel("Coefficient (impact on Price, all else held constant)")
    plt.title("Linear Regression Coefficients")
    plt.gca().invert_yaxis()
    plt.tight_layout()
```

```
plt.show()

print(f"Intercept: {lr.intercept_:.2f}")
```



Intercept: 99,339.49

Interpreting coefficients: each coefficient is the expected change in `Price` for a one-unit increase in that feature, holding all other features constant. For example, the `Area_sqft` coefficient is roughly "extra dollars per extra square foot." Positive bars (green) push price up; negative bars (red) push price down. Because features are on different scales (square feet vs. binary garage vs. years of age), don't compare coefficient *magnitudes* directly across features unless you've standardized them first — a large coefficient on a small-scale feature (like `Garage`, which is just 0/1) can still represent a meaningful real-world effect.

12. Bonus: Ridge & Lasso Regularization Comparison

Regularized regression adds a penalty on coefficient size, which can help when features are correlated (multicollinearity) or when the model is overfitting. `Ridge` shrinks coefficients smoothly; `Lasso` can shrink some coefficients to exactly zero, effectively performing feature selection.

```
In [21]: models = {
    "Linear Regression": lr,
    "Ridge (alpha=1.0)": Ridge(alpha=1.0, random_state=RANDOM_STATE),
    "Lasso (alpha=1.0)": Lasso(alpha=1.0, random_state=RANDOM_STATE, max_
}

results = []
for name, model in models.items():
    if name != "Linear Regression":
        model.fit(X_train, y_train)
```

```

preds = model.predict(X_test)
results.append({
    "Model": name,
    "MSE": mean_squared_error(y_test, preds),
    "RMSE": np.sqrt(mean_squared_error(y_test, preds)),
    "R2": r2_score(y_test, preds),
})

results_df = pd.DataFrame(results).set_index("Model").round(3)
results_df

```

Out[21]:

	MSE	RMSE	R2
Model			

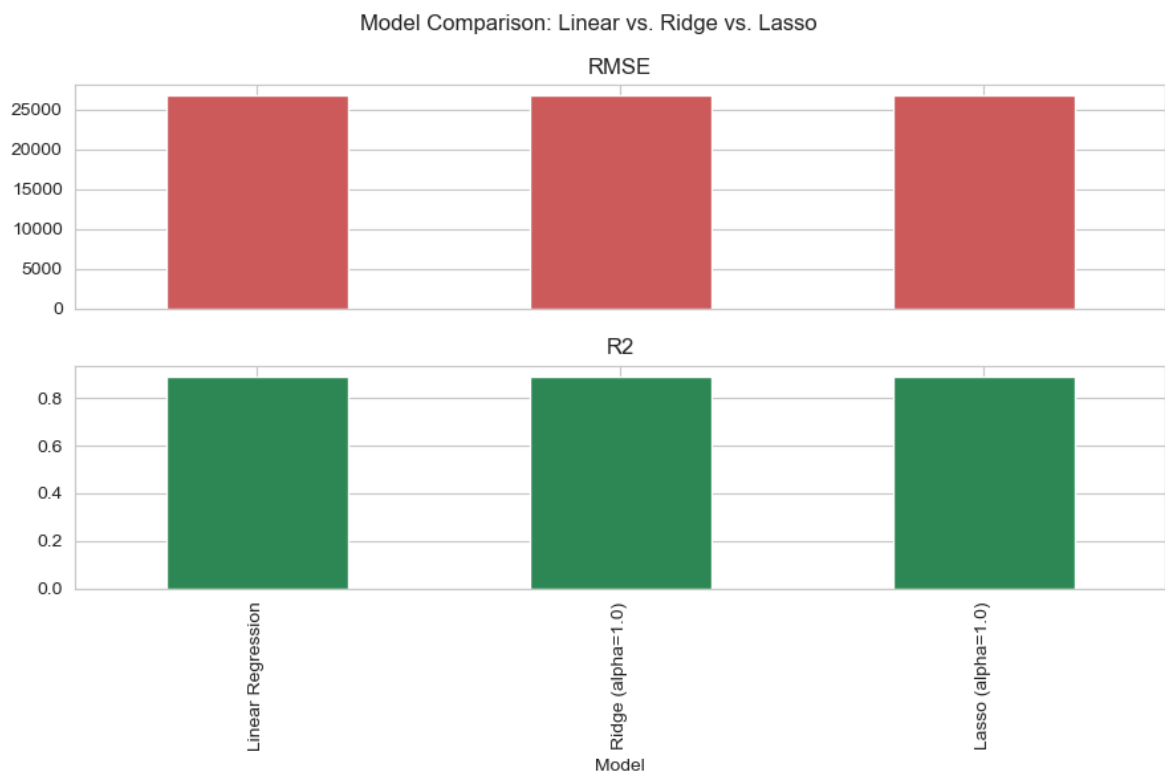
Model	MSE	RMSE	R2
Linear Regression	7.122052e+08	26687.173	0.892
Ridge (alpha=1.0)	7.126247e+08	26695.031	0.892
Lasso (alpha=1.0)	7.122105e+08	26687.272	0.892

In [22]:

```

results_df[["RMSE", "R2"]].plot(kind="bar", subplots=True, figsize=(9, 6))
plt.suptitle("Model Comparison: Linear vs. Ridge vs. Lasso")
plt.tight_layout()
plt.show()

```



Takeaway: on a dataset like this, with only a handful of features and a genuinely linear underlying relationship, plain Linear Regression, Ridge, and Lasso will usually perform very similarly — regularization mainly earns its keep when you have many features and/or strong multicollinearity, where it stabilizes coefficient estimates and can prevent overfitting. If you swap in the real Ames dataset (~80 columns), you should see Ridge/Lasso pull ahead of plain linear regression more noticeably, and Lasso's feature-zeroing behavior becomes genuinely useful for trimming down the

feature set. Try sweeping `alpha` over `[0.01, 0.1, 1, 10, 100]` with `RidgeCV` / `LassoCV` to tune it properly.

13. Summary

- Cleaned missing values (median for numeric, mode for categorical) and one-hot encoded `Location`.
- Explored the correlation structure and confirmed `Area_sqft` and `Bathrooms` / `Bedrooms` are the strongest positive predictors, `Age_years` / `Distance_to_City_km` are negative predictors, and `Location` matters as expected.
- Trained a Linear Regression model, evaluated it with MSE/RMSE/ R^2 , and visually confirmed reasonably random residuals.
- Compared against Ridge and Lasso as a regularization sanity-check.

In []: